> **Avertissement :** ce rapport décrit une preuve de concept technique.
> Les résultats présentés ne constituent pas une validation clinique et ne permettent pas, à eux seuls, d'autoriser l'utilisation du système dans la prise en charge réelle de patients.

---

1. Résumé exécutif

1.1 Contexte

Le projet vise à étudier la faisabilité d'un agent d'intelligence artificielle capable d'assister un professionnel dans des tâches de réponse à des questions médicales et, à terme, dans un contexte de triage.

L'objectif du POC est de valider la faisabilité d'une chaîne complète et reproductible permettant :

- de préparer un corpus médical anonymisé ;
- d'entraîner un modèle de langage compact ;
- de spécialiser son comportement par Supervised Fine-Tuning (SFT) ;
- d'optimiser ses préférences par Direct Preference Optimization (DPO) ;
- d'évaluer objectivement différentes versions du modèle ;
- de déployer le modèle localement ;
- d'exposer le modèle via une API ;
- de préparer une architecture reproductible pouvant évoluer vers un déploiement à plus grande échelle.

Le projet privilégie une architecture locale afin de conserver la maîtrise de l'environnement d'exécution et des données.

1.2 Approche retenue

Le modèle de base retenu est :
Qwen/Qwen3-1.7B-Base

Le pipeline suit la chaîne :

```
Données médicales
       ↓
Préparation / anonymisation
       ↓
Split reproductible
       ↓
SFT LoRA
       ↓
DPO
       ↓
Évaluation indépendante
       ↓
Sélection du modèle
       ↓
Docker
       ↓
vLLM
       ↓
FastAPI
       ↓
API de service
```

1.3 Résultats principaux

Indicateur	Résultat
Nombre d'exemples SFT	5000
Train	4000
Validation	500
Test	500
Benchmark	776 exemples

Indicateur	Résultat
Base	31,57 %
Meilleur SFT v3	34,02 %
DPO v8	61,60 %

Les résultats montrent une amélioration importante entre le modèle de base et les modèles optimisés par DPO sur le benchmark de préférence utilisé.

Cette amélioration doit toutefois être interprétée comme **une amélioration expérimentale sur le benchmark**, et non comme une preuve d'exactitude ou de sécurité clinique.

1.4 Conclusion principale

Le POC démontre la faisabilité technique d'une chaîne locale complète de spécialisation, d'évaluation et de déploiement d'un modèle médical compact.

Le passage à un usage opérationnel au CHSA nécessiterait cependant une phase supplémentaire de validation portant notamment sur :

- l'exactitude médicale ;
- la sécurité ;
- les erreurs critiques ;
- la robustesse ;
- la validation par des professionnels de santé ;
- la protection des données ;
- la traçabilité ;
- la gouvernance du modèle.

---

2. Contexte et objectifs

2.1 Contexte

L'utilisation de modèles de langage dans le domaine médical présente un potentiel important pour l'assistance aux professionnels, notamment pour :

- la recherche d'information ;
- la synthèse ;
- l'aide à l'interprétation ;
- la génération de réponses ;
- l'assistance au triage ;
- l'orientation vers des ressources médicales.
  Cependant, le domaine médical impose des contraintes particulières.
  Une réponse grammaticalement correcte ou jugée pertinente par un benchmark automatique peut être médicalement incorrecte ou dangereuse.
  Le projet P14 doit donc être considéré comme une démarche expérimentale visant d'abord à démontrer la faisabilité technique.

2.2 Objectifs techniques

Les objectifs sont :

1. constituer un corpus exploitable ;
2. rendre sa préparation reproductible ;
3. entraîner un modèle compact localement ;
4. comparer plusieurs stratégies de fine-tuning ;
5. mesurer l'effet du SFT ;
6. mesurer l'effet du DPO ;
7. mettre en place une évaluation indépendante ;
8. déployer le modèle ;
9. mesurer les performances d'inférence ;
10. préparer une architecture évolutive.

2.3 Objectifs non couverts

Le POC ne vise pas à démontrer :

- la sécurité clinique ;
- la conformité réglementaire complète ;
- la capacité à remplacer un professionnel ;
- la capacité à prendre seul une décision médicale ;
- la performance sur des patients réels ;
- la validation prospective en environnement clinique.

---

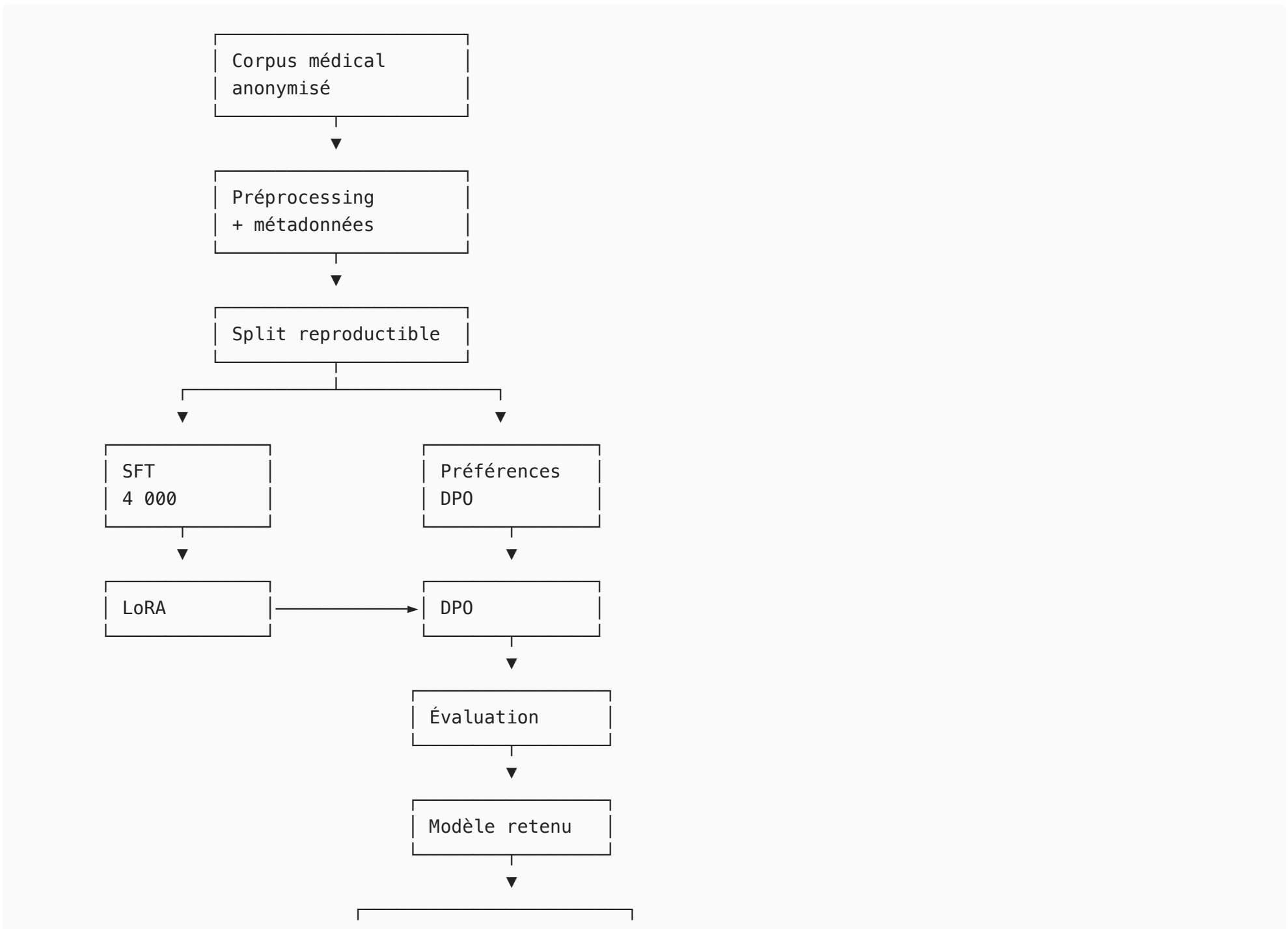
3. Périmètre du POC

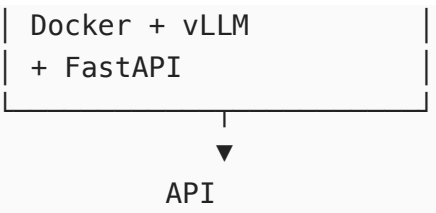
3.1 Périmètre inclus /exclu

Le projet couvre :	Le projet ne couvre pas encore :
- préparation des données ; - preprocessing ; - split reproductible ; - SFT ; - LoRA ; - DPO ; - évaluation ; - analyse comparative ; - conteneurisation ; - inférence GPU ; - API ; - documentation ; - reproductibilité.	- la mise en production hospitalière ; - l'autonomie décisionnelle ; - l'intégration complète au SIH ; - la validation réglementaire ; - la validation clinique prospective ; - l'utilisation auprès de patients.

4. Architecture technique

4.1 Architecture générale





4.2 Composants

Composant	Rôle
Python	Environnement d'exécution
PyTorch	Entraînement
Transformers	Modèle et tokenizer
TRL	SFT / DPO
PEFT	LoRA
Unsloth	Optimisation de l'entraînement
Datasets	Gestion des datasets
bitsandbytes	Optimisations mémoire
vLLM	Inférence
FastAPI	API
Docker	Conteneurisation
GitHub Actions	CI/CD
uv	Gestion de l'environnement

5. Environnement matériel et logiciel pour le POC

5.1 Matériel

PC portable local avec NVIDIA RTX 5080 Laptop VRAM 16 Go

5.2 Environnement logiciel

```
Python       : 3.14
PyTorch      : 2.14
CUDA         : 13.0
TRL          : 0.24.0
Unsloth      : 2025.7.2
```

Les versions exactes utilisées sont dans `pyproject.toml` et `uv.lock` (suite à une une migration du projet entre 2 pc, un conflit existe entre les versions, utiliser wsl2 pour) .

6. Préparation des données

6.1 Sources

Le corpus SFT comprend environ 5 000 exemples.

Source	Nombre
FrenchMedMCQA	1 667
MediQAl	1 666
MedQuAD	1 667
Total	**5 000**

6.2 Préprocessing

Les opérations comprennent :

- normalisation ;
- vérification des champs ;
- harmonisation du format ;

- conservation de la provenance ;
- ajout de métadonnées ;
- contrôle de la qualité ;
- préparation du format instruction/réponse.

6.3 Métadonnées

Les données peuvent notamment conserver :

- contexte clinique ;
- source ;
- provenance ;
- qualité ;
- version du preprocessing.

Ces informations permettent d'assurer la traçabilité des données utilisées.

6.4 Anonymisation

Les données utilisées dans le pipeline doivent être anonymisées.

Aucune donnée de santé directement identifiable ne doit être intégrée :

- au dépôt Git ;
- aux logs ;
- aux résultats ;
- aux benchmarks publics ;
- aux artefacts de déploiement.

L’anonymisation est réalisée automatiquement avec **Microsoft Presidio**, associé aux modèles linguistiques spaCy français et anglais.

- scripts:
  - /scripts/anonymize_dpo.py
  - /scipts/anonymize_sft.py
- rapports :
  - /reports/
    - pii_report.json
    - pii_reports.txt
    - predisio_aninymisation_reports.json
    - predisio_aninymisation_reports.txt

Les principales entités recherchées sont :

Entité	Remplacement
`PERSON`	`[PERSON]`
`EMAIL_ADDRESS`	`[EMAIL]`
`PHONE_NUMBER`	`[PHONE]`
`IP_ADDRESS`	`[IP]`
`LOCATION`	`[LOCATION]`
`MEDICAL_LICENSE`	`[MEDICAL_LICENSE]`

La stratégie utilisée est le remplacement (`replace`).

7. Stratégie de séparation des données

Le split principal est :

```
Train       : 4 000
Validation  :   500
Test        :   500
Total       : 5 000
Seed        : 3407
```

Le split doit être déterministe afin de permettre la reproduction exacte des expériences.
Le jeu de test doit rester séparé de l'entraînement.
Le benchmark RewardBench utilisé pour la comparaison des modèles constitue une évaluation distincte.
(split effectué par le script split_sft,py)

- **Seed :** 3407, pour garantir la reproductibilité.
- **Méthode de split :** séparation **80 % / 10 % / 10 %** en train, validation et test.
- **Stratification :** split stratifié avec quotas proportionnels et méthode des plus grands restes.
- **Critères de stratification :** combinaison **source du dataset + langue + type de tâche**.
- **Absence de fuite :** contrôle des IDs et des couples **instruction + réponse** entre les trois ensembles.
- **Reproductibilité :** mélange déterministe avec la seed 3407 et génération d'une empreinte (`fingerprint`) des trois splits.
- **Version du dataset :** `sft_5000_anonymized.jsonl` , dataset source de **5 000 exemples**.

8. Format d'entraînement

Les exemples sont structurés autour d'une instruction et d'une réponse attendue.
Le prompt impose notamment :

- réponse dans la langue de la question ;
- réponse centrée sur la question ;
- absence d'informations hors sujet ;
- absence de faits inventés ;
- reconnaissance de l'incertitude lorsque les informations sont insuffisantes.

L'entraînement utilise une loss appliquée à la partie réponse/complétion.
Cette approche évite de traiter l'instruction comme une cible à apprendre.

9. Supervised Fine-Tuning

9.1 Modèle

```
Qwen/Qwen3-1.7B-Base
```

Le choix d'un modèle de petite taille répond à plusieurs objectifs :

- entraînement local ;
- consommation mémoire maîtrisée ;
- expérimentation rapide ;
- déploiement relativement léger ;
- possibilité d'utiliser une infrastructure GPU limitée.

9.2 LoRA

Le fine-tuning est réalisé avec LoRA.

Configuration de référence :

Paramètre	**Valeur**
Rank	32
Alpha	64
Dropout	0
Learning rate	$2{,}8 \times 10^{-4}$ de base puis variation sur les versions
Max sequence length	2048
Batch size	1
Gradient accumulation	8
Epochs	1 à 5 pour les 1ers modèles puis 1 à 3
Seed	3407

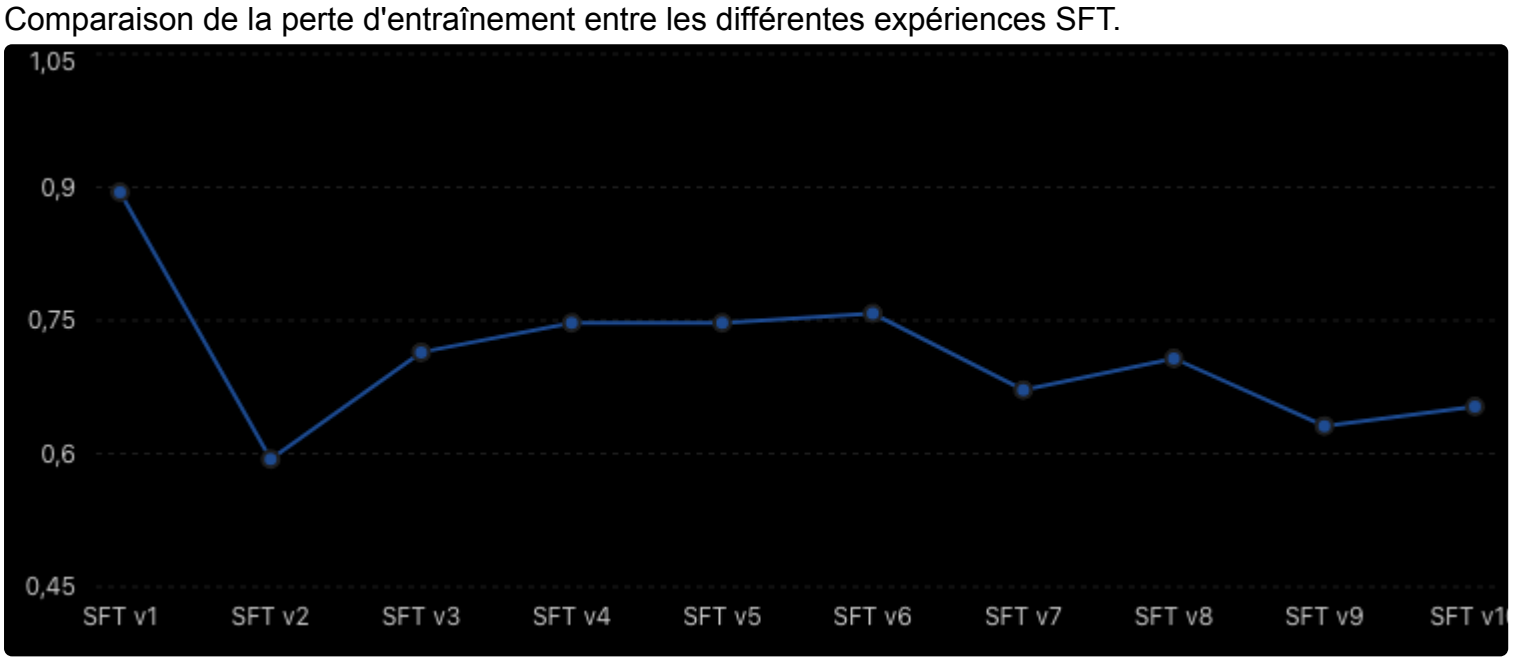
9.3 Résultats SFT

Modèle	Train loss	Eval loss	Runtime	RewardBench
SFT v1	0894526903629302	1.1658194065093994	37.9678	33,76 %
SFT v2	0.5937519798278809	0.7754403352737427	38.7926	33,51 %
SFT v3	0.7141744441986084	0.7261147499084473	38.7183	**34,02 %**
SFT v4	0.747199369430542	0.45074138045310974	38.9827	31,31 %
SFT v5	0.747199369430542	0.45074138045310974	38.443	31,31 %
SFT v6	0.7577322982152304	0.4501045048236847	41.4797	31,31 %
SFT v7	0.6717159531911214	0.4551416337490082	38.5505	32,60 %
SFT v8	0.7072962266604106	0.4493294060230255	39.0997	30,54 %
SFT v9	0.6312958909670512	0.4642016589641571	38.4586	31,70 %
SFT v10	0.6531015267372131	0.4747532904148102	39.4778	32,35 %

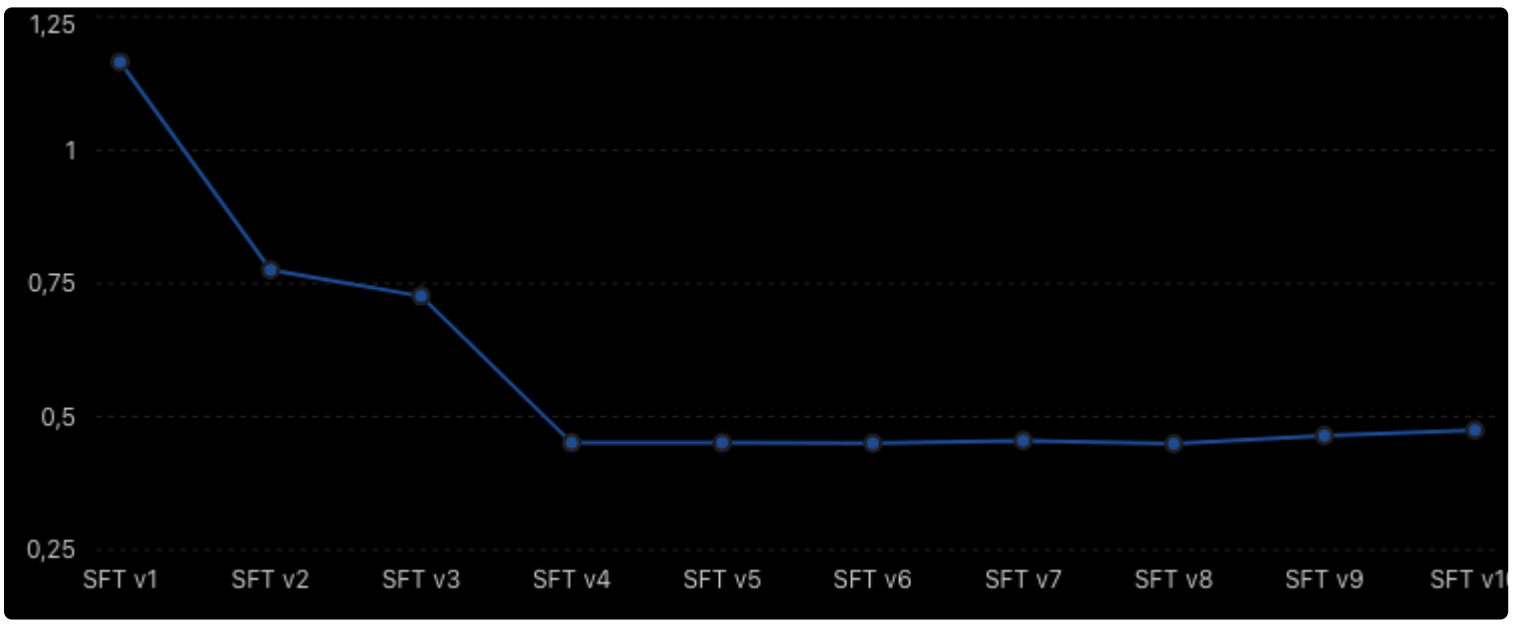
Analyse

Le SFT seul produit une amélioration relativement limitée sur le benchmark considéré.
Cela suggère que l'apprentissage supervisé améliore certaines propriétés de génération sans nécessairement optimiser directement la préférence mesurée par RewardBench.

Évolution de la Train Loss — SFT v1 à v10
Comparaison de la perte d'entraînement entre les différentes expériences SFT.



Évolution de l'Eval Loss — SFT v1 à v10
Comparaison de la perte de validation entre les différentes expériences SFT.



- **Eval loss** chute fortement entre SFT v1 et SFT v4.
- Elle reste ensuite très stable autour de **0,45–0,47**.
- Le meilleur `eval_loss` est **SFT v8 : 0,4493**.
- Pourtant, **SFT v8 n'a pas le meilleur RewardBench** : 30,54 %.
- Le meilleur RewardBench SFT est **SFT v3 : 34,02 %**, avec une `eval_loss` de 0,7261.

---

10. Direct Preference Optimization

10.1 Principe

Le DPO vise à apprendre une préférence entre deux réponses :

```
Prompt
   ├── Chosen
   └── Rejected
```

Le modèle est ainsi entraîné à augmenter la probabilité de la réponse préférée par rapport à la réponse rejetée.

10.2 Données

Les données DPO sont constituées de paires :

```
prompt
chosen
rejected
```

Les paires sont issues du jeu de données **UltraMedical-Preference**. Le pipeline de préparation ne génère donc pas de nouvelles réponses médicales : il sélectionne et prépare des préférences existantes pour leur utilisation dans l'entraînement DPO.

La construction du jeu de données suit plusieurs étapes.
1. Validation structurelle des exemples
2. Filtrage des types de préférences
3. Vérification de la cohérence de la préférence
4. Sélection selon la qualité documentaire de la préférence
5. Prévention des fuites de données
**6. Sélection stratifiée
La sélection est reproductible grâce à une graine aléatoire fixée (`seed = 42` par défaut).

Le dataset final contient :

- **5 000 exemples pour l'entraînement DPO** ;
- **1 000 exemples pour la validation DPO**.

7. Séparation train / validation au niveau du prompt
8. Métadonnées et traçabilité
Le dataset final conserve les informations nécessaires à la traçabilité :

```
id
prompt
chosen
rejected
source
quality
provenance
```

10.3 Expériences

Version	SFT source	Learning rate	Beta	Accuracy	autre
DPO v1	sft_v3	1e-5	0,1	34,54 %	Modification du script
DPO v2	sft_v3	1e-5	0,1	33,51 %	Modification du script
DPO v3	sft_v3	1e-05	0,1	38,40 %	Modification du script
DPO v4	sft_v3	1e-05	0,1	42,53 %	Script v2
DPO v5	sft_v3	1e-05	0,2	39,30 %	Script v2
DPO v6	sft_v3	2e-05	0,1	44,97 %	Script v2
DPO v7	sft_v3	0.0001	0,1	52,32 %	Script v2
DPO v8	sft_v3	0.0005	0,1	**61,60 %**	Script v2
DPO v9	sft_v3	0.001	0,1	61,08 %	Script v2
DPO v10	sft_v3	0.0007	0,1	61,47 %	Script v2

10.4 Analyse

L'évolution montre une progression particulièrement importante à partir des expériences DPO intermédiaires.

Le meilleur score observé est :
DPO v8 : 478 / 776 = 61,60 %.

11. Protocole d'évaluation

11.1 Benchmark

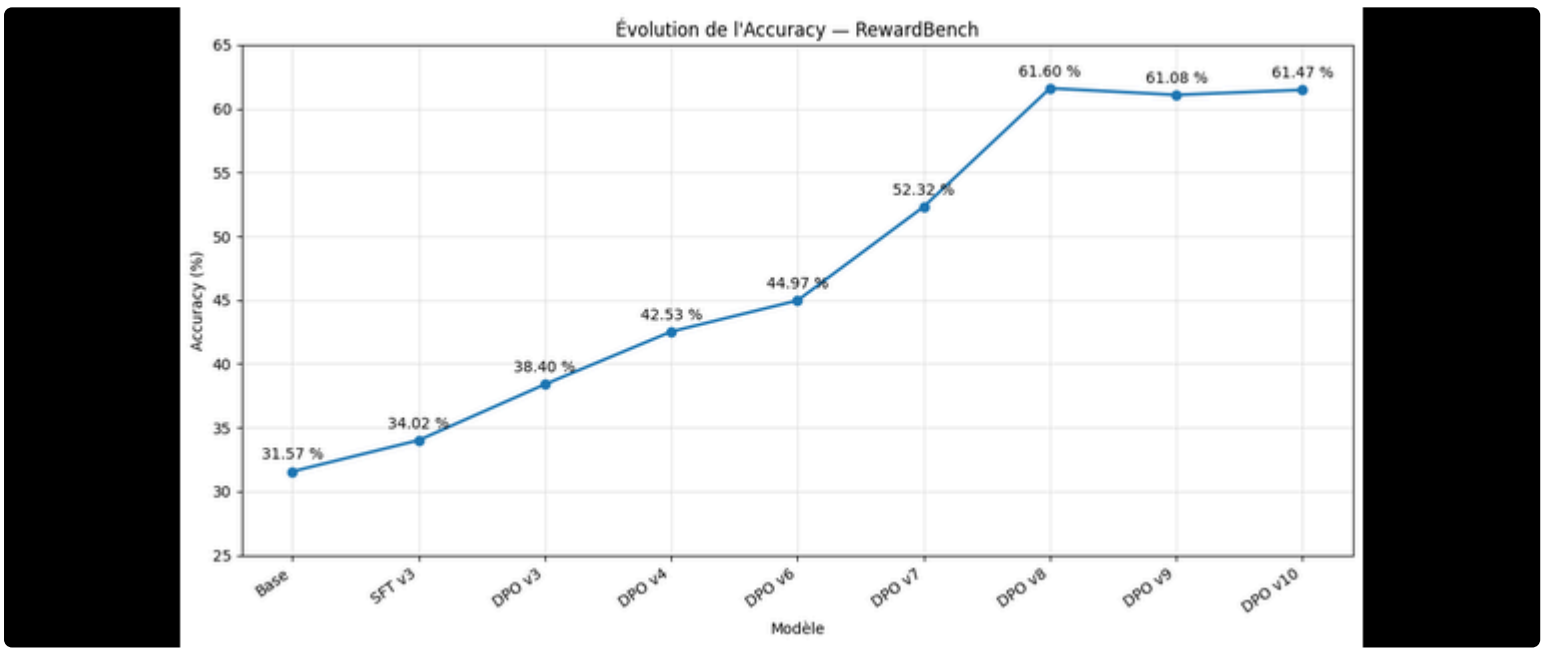
Le benchmark utilisé comprend :
776 exemples.
L'évaluation compare la préférence du modèle entre une réponse `chosen` et une réponse `rejected`.

11.2 Métrique principale

```
Accuracy =
nombre de préférences correctes
/
nombre total de préférences
```

11.3 Résultats

Modèle	Correct	Total	Accuracy
Base	245	776	31,57 %
SFT v3	264	776	34,02 %
DPO v3	298	776	38,40 %
DPO v4	330	776	42,53 %
DPO v6	349	776	44,97 %
DPO v7	406	776	52,32 %
DPO v8	**478**	776	**61,60 %**
DPO v9	474	776	61,08 %
DPO v10	477	776	61,47 %



12. Limites de l'évaluation

RewardBench ne mesure pas directement :

- l'exactitude clinique ;
- la sécurité ;
- la pertinence dans une situation de triage réelle ;
- la capacité à identifier une urgence ;
- la capacité à refuser une réponse dangereuse ;
- la robustesse face à des informations contradictoires ;
- la qualité de la prise de décision clinique.

Il ne faut donc pas assimiler :

```
RewardBench accuracy
```

à :

```
Clinical accuracy
```

Cette distinction est fondamentale.

13. Évaluation qualitative et clinique

13.1 Limites des métriques lexicales

Pour une réponse médicale générative, plusieurs formulations peuvent être correctes.

Les métriques telles que :

- Exact Match ;
- token-F1 ;
- similarité lexicale
- script d'évaluation des qa et qcm
  sont donc insuffisantes comme métriques principales.

13.2 Évaluation par LLM

Une évaluation automatique par LLM peut être utilisée comme outil exploratoire.
Elle ne doit toutefois pas être considérée comme une validation clinique indépendante.

Ses limites comprennent notamment :

- biais du modèle juge ;
- corrélation possible entre modèle générateur et modèle juge ;
- difficulté à détecter certaines erreurs médicales ;
- sensibilité au prompt ;
- absence de responsabilité clinique.

13.3 Évaluation recommandée

Pour une future phase de validation :

```
Évaluation automatique
        +
Analyse qualitative
        +
Experts médicaux
        +
Cas difficiles
        +
Tests de sécurité
        +
Analyse des erreurs
```

14. Analyse / Benchmarks du contenu des réponses

Les Erreurs de factualité, Hallucinations, Omission d'informations importantes, Mauvaise interprétation de la question, Réponses excessivement longues ont été mesuré de différentes manières tout au long de l 'entrainement :
Le même jeu de test, test_sft.json a servi de base aux script comparatif de génération de prédiction:

- f1 score
- exact match
- llm Juge (minimistral) sachant qu'un llm juge de base comparé à un groupe d'expert à 4/11 de fiabilité

Détail FR / EN

Modèle	Langue	Faithfulness	Relevancy	Correctness	Completeness	GLOBAL
Base	EN	2,529	3,206	2,588	**2,235**	**2,640**
Base	FR	1,864	2,364	2,152	1,455	1,958
SFT_v5	EN	2,353	**3,294**	2,588	1,882	2,529

Modèle	Langue	Faithfulness	Relevancy	Correctness	Completeness	GLOBAL
SFT_v5	FR	1,652	2,742	1,879	1,621	1,973
DPO_v1	EN	2,412	3,147	2,735	1,882	2,544
DPO_v1	FR	1,621	2,894	1,742	1,697	1,989
DPO_v2	EN	2,559	**3,353**	2,735	**1,912**	**2,640**
DPO_v2	FR	1,682	2,742	**1,833**	1,591	1,962
DPO_v3	EN	2,559	3,176	**2,765**	1,794	2,574
DPO_v3	FR	1,621	2,803	1,697	1,606	1,932

15. Déploiement

15.1 Docker

Le modèle est conteneurisé afin de reproduire l'environnement d'inférence.

```
Docker
   ↓
vLLM
   ↓
Qwen3-1.7B
   +
LoRA
```

15.2 vLLM

vLLM est utilisé comme moteur d'inférence.

Cette séparation permet de distinguer :

```
Entraînement
      ≠
Évaluation
      ≠
Inférence
```

15.3 FastAPI

FastAPI fournit l'interface applicative.

L'API permet notamment de retourner :

- l'identifiant de requête ;
- le modèle utilisé ;
- la réponse générée.

Le `request_id` constitue un premier mécanisme de traçabilité.

15.4 Documentation

La documentation interactive est accessible via :

```
/docs
```

16. Reproductibilité

La reproductibilité repose sur :

- seed ;
- split déterministe ;
- datasets versionnés ;
- configuration enregistrée ;
- checkpoints ;
- résultats sauvegardés ;

- `pyproject.toml` ;
- `uv.lock` ;
- Docker ;
- scripts automatisés ;
- Git.

Chaque expérience doit pouvoir être identifiée par :

```
Experiment ID
Base model
Dataset
Dataset version
Preprocessing version
Seed
Hyperparameters
Checkpoint
Evaluation version
Result
```

17. Traçabilité

La chaîne de traçabilité cible est :

```
Dataset
   ↓
Dataset version
   ↓
Preprocessing
   ↓
SFT
   ↓
Checkpoint
   ↓
DPO
   ↓
Evaluation
   ↓
Model version
   ↓
Deployment
   ↓
Inference
   ↓
Request ID
```

Cette traçabilité est particulièrement importante dans un contexte médical.

18. Sécurité et protection des données

Le passage à l'échelle doit prendre en compte :

Données	API
- anonymisation ; - minimisation ; - contrôle d'accès ; - chiffrement ; - conservation limitée ; - suppression sécurisée.	- authentification ; - autorisation ; - limitation du débit ; - validation des entrées ; - protection contre les abus.

Logs

Les logs ne doivent pas contenir inutilement de données médicales.
Ils doivent privilégier :

```
request_id
timestamp
```

```
model_version
latency
status
error
```

plutôt que le stockage systématique du contenu médical.

19. Limites actuelles

Les principales limites identifiées sont :

1. taille réduite du modèle ;
2. volume limité du corpus ;
3. couverture médicale incomplète ;
4. dépendance aux données de préférence ;
5. possibilité d'hallucinations ;
6. absence de validation clinique prospective ;
7. absence de validation sur patients réels ;
8. limites du benchmark automatique ;
9. absence de garantie de sécurité clinique ;
10. absence éventuelle de RAG ;
11. absence d'intégration complète au SIH ;
12. absence de validation réglementaire complète.
13. Pas de méthodologie de triage définie par le personnel médical

20. Analyse stratégique

20.1 Forces

- architecture locale ;
- faible taille du modèle ;
- coût matériel maîtrisé ;
- pipeline reproductible ;
- SFT + DPO ;
- déploiement Docker ;
- vLLM ;
- FastAPI ;
- traçabilité ;
- benchmark indépendant ;
- possibilité d'itération rapide.

20.2 Faiblesses

- corpus limité ;
- benchmark non clinique ;
- modèle compact ;
- absence de validation experte systématique ;
- couverture des cas médicaux limitée.

20.3 Opportunités

- constitution d'un dataset CHSA ;
- annotation médicale ;
- RAG ;
- human-in-the-loop ;
- intégration au SIH ;
- monitoring ;
- amélioration continue ;
- spécialisation par service.

20.4 Menaces

- hallucinations ;
- mauvaises décisions ;
- biais ;
- fuite de données ;
- dérive du modèle ;
- mauvaise interprétation des résultats ;
- utilisation hors du périmètre prévu.

21. Roadmap de passage à l'échelle

Phase 0 — POC

Statut : réalisé

```
Dataset
SFT
DPO
Evaluation
Docker
vLLM
FastAPI
```

Objectif :

> démontrer la faisabilité technique.

Phase 1 — Validation contrôlée

Objectif :

> déterminer si le système présente une valeur suffisante pour poursuivre.

Actions :

- dataset CHSA plus représentatif ;
- cas cliniques difficiles ;
- annotation par professionnels ;
- benchmark verrouillé ;
- évaluation experte ;
- analyse des erreurs ;
- tests de sécurité ;
- mesure systématique de latence.

Critère de sortie :
[À définir avec les équipes médicales.]

Phase 2 — Pilote interne

Déploiement limité à un groupe de professionnels.

Architecture :

```
Professionnel
      ↓
Interface
      ↓
API
      ↓
Guardrails
      ↓
LLM
      ↓
Réponse
```

```
      ↓
Validation humaine
```

Le modèle ne doit pas prendre seul la décision médicale.

Phase 3 — Préproduction

Actions :

- monitoring ;
- authentification ;
- gestion des utilisateurs ;
- audit ;
- sauvegardes ;
- haute disponibilité ;
- tests de charge ;
- procédure de rollback ;
- gestion des versions ;
- gestion des incidents.

Phase 4 — Passage à l'échelle

Possibilité d'architecture :



22. Évolution fonctionnelle recommandée

22.1 RAG

Une évolution importante consiste à compléter le modèle par une récupération documentaire.

```
Question
   ↓
Recherche documentaire
   ↓
Sources médicales validées
   ↓
Contexte
   ↓
LLM
```

```
   ↓
Réponse sourcée
```

Cela permet de réduire la dépendance à la connaissance mémorisée dans les poids du modèle.

22.2 Human-in-the-loop

Le professionnel doit rester dans la boucle :

```
IA
 ↓
Proposition
 ↓
Professionnel
 ↓
Validation / correction
 ↓
Décision
```

22.3 Amélioration continue

Les corrections humaines peuvent ensuite servir à construire :

- nouvelles données SFT ;
- nouvelles préférences DPO ;
- jeux d'évaluation ;
- cas adversariaux.

23. Matrice de risques

Risque	Gravité	Probabilité	Mitigation
Hallucination médicale	Critique	[ ]	RAG + validation humaine
Mauvais triage	Critique	[ ]	Human-in-the-loop
Fuite de données	Critique	[ ]	Sécurité + anonymisation
Réponse obsolète	Élevée	[ ]	Sources versionnées
Biais	Élevée	[ ]	Evaluation par sous-groupes
Indisponibilité	Élevée	[ ]	Monitoring + redondance
Dégradation du modèle	Élevée	[ ]	Benchmark de non-régression
Prompt injection	Élevée	[ ]	Guardrails
Traçabilité insuffisante	Élevée	[ ]	Audit logs
Mauvaise utilisation	Critique	[ ]	Gouvernance + formation

24. Recommandations stratégiques

Recommendation 0 - Définir la métodologie de triage

Méthode	Pays / contexte principal	Niveaux	Ce qu'elle fait principalement
ESI — Emergency Severity Index	🇺🇸 États-Unis	**5**	Classe le patient selon **gravité/risque** puis, pour les patients moins graves, selon les **ressources nécessaires**.
CTAS — Canadian Triage and Acuity Scale	🇨🇦 Canada	**5**	Classe l'acuité à partir du motif, de l'histoire, des signes cliniques et de critères spécifiques ; sert notamment à déterminer la priorité de prise en charge.
ATS — Australasian Triage Scale	🇦🇺 Australie / 🇳🇿 Nouvelle-Zélande	**5**	Classe surtout selon **l'urgence clinique**, avec des descripteurs physiologiques, le risque et la gravité des symptômes.
MTS — Manchester Triage System	🇬🇧 Royaume-Uni et nombreux pays européens	**5**	Utilise le **motif de consultation**, puis un arbre décisionnel avec des critères discriminants pour déterminer la catégorie de priorité.
Échelles à 3 niveaux	Historiquement utilisées dans plusieurs pays	**3**	Typiquement **urgent / semi-urgent / non urgent** ou équivalent. Plus simples, mais moins discriminantes que les systèmes modernes à 5 niveaux.

Recommandation 1 — Maintenir le POC en environnement local

L'architecture locale actuelle constitue une bonne base pour continuer les expérimentations tout en conservant la maîtrise de l'environnement.

Recommandation 2 — Ne pas considérer RewardBench comme une validation clinique

RewardBench doit rester un indicateur expérimental parmi plusieurs autres.

Recommandation 3 — Construire un benchmark clinique CHSA

Le prochain benchmark devrait contenir des cas représentatifs du contexte réel et être évalué par plusieurs professionnels.

Recommandation 4 — Mettre en place un jeu de test verrouillé

Le jeu de test ne doit jamais être utilisé pour l'entraînement.
Il doit devenir la référence de non-régression des futures versions.

Recommandation 5 — Introduire progressivement le RAG

Les connaissances médicales évoluant dans le temps, le système doit pouvoir s'appuyer sur des sources contrôlées et actualisées.

Recommandation 6 — Conserver un humain dans la boucle

Le système doit être présenté comme un outil d'assistance et non comme un décideur autonome.

Recommandation 7 — Mettre en place une gouvernance des modèles

Chaque version doit être associée à :

```
Model ID
Dataset version
Training configuration
Evaluation results
Approval status
Deployment status
Rollback version
```

Recommandation 8 — Définir des critères de passage entre les phases

Le passage POC → pilote → production ne doit pas être basé uniquement sur une métrique de benchmark.

25. Critères de décision pour un passage au pilote

Avant tout pilote, il est recommandé de définir des critères mesurables.

Modèle

- performance minimale au benchmark ;
- absence d'augmentation des erreurs critiques ;
- seuil de factualité ;
- seuil de robustesse.

Clinique

- validation par professionnels ;
- analyse des cas dangereux ;
- validation des scénarios critiques.

Système

- P95 de latence ≤ **[X ms]** ;
- disponibilité ≥ **[X %]** ;
- taux d'erreur ≤ **[X %]** ;
- capacité de charge ≥ **[X requêtes/minute]**.

Sécurité

- absence de données sensibles dans les logs ;

- authentification ;
- contrôle d'accès ;
- audit.

26. Indicateurs de suivi futurs

Un système opérationnel devrait suivre au minimum :

Domaine	KPI
Modèle	Accuracy benchmark
Clinique	Taux d'erreurs critiques
Clinique	Taux d'omission
Clinique	Taux d'hallucination
Système	P50
Système	P95
Système	P99
Système	Tokens/s
Système	Disponibilité
Infrastructure	VRAM
Infrastructure	CPU/RAM
Sécurité	Incidents
Usage	Nombre de requêtes
Qualité	Corrections humaines
Gouvernance	Version du modèle

28. Conclusion

Le projet P14 démontre la faisabilité technique d'un pipeline local complet pour la spécialisation et le déploiement d'un modèle de langage compact dans un contexte médical.

Le pipeline couvre l'ensemble de la chaîne :

```
Données
   ↓
Préprocessing
   ↓
Split reproductible
   ↓
SFT LoRA
   ↓
DPO
   ↓
Evaluation
   ↓
Sélection
   ↓
Docker
   ↓
vLLM
   ↓
FastAPI
   ↓
CI/CD
```

Les expérimentations montrent une amélioration significative de la performance mesurée sur le benchmark de préférence utilisé, avec un meilleur score de **61,60 % pour DPO v8**, contre **31,57 % pour le modèle de base**.

Cette progression constitue un résultat technique intéressant, mais ne permet pas de conclure à la sécurité ou à l'exactitude clinique du système.

La prochaine étape recommandée n'est donc pas un déploiement clinique direct, mais une **phase de validation contrôlée**, reposant sur :

1. un dataset représentatif du CHSA ;
2. une évaluation par des professionnels de santé ;
3. un benchmark clinique indépendant ;
4. des tests de sécurité et de robustesse ;
5. une mesure complète des performances système ;
6. une architecture human-in-the-loop ;
7. une gouvernance stricte des versions du modèle.

Le POC constitue ainsi une **base technique pour une éventuelle expérimentation clinique contrôlée**, mais ne doit pas être considéré comme un système médical prêt pour la production.